1 - Introduzione e concetti generali

Tutti i sistemi software possono presentare delle difformità rispetto alle attese dell'utilizzatore, dovute sia in parte all'utilizzatore, che non ne capisce l'utilizzo, sia a difetti stessi del sistema, dovuti ad errori avvenuti nelle fasi di analisi o progettazione.

Si vogliono introdurre delle garanzie per fare in modo che le richieste del cliente siano rispettate nel prodotto finale.

Qualità dei sistemi software

Assicurare la qualità significa effettuare verifica e validazione del prodotto:

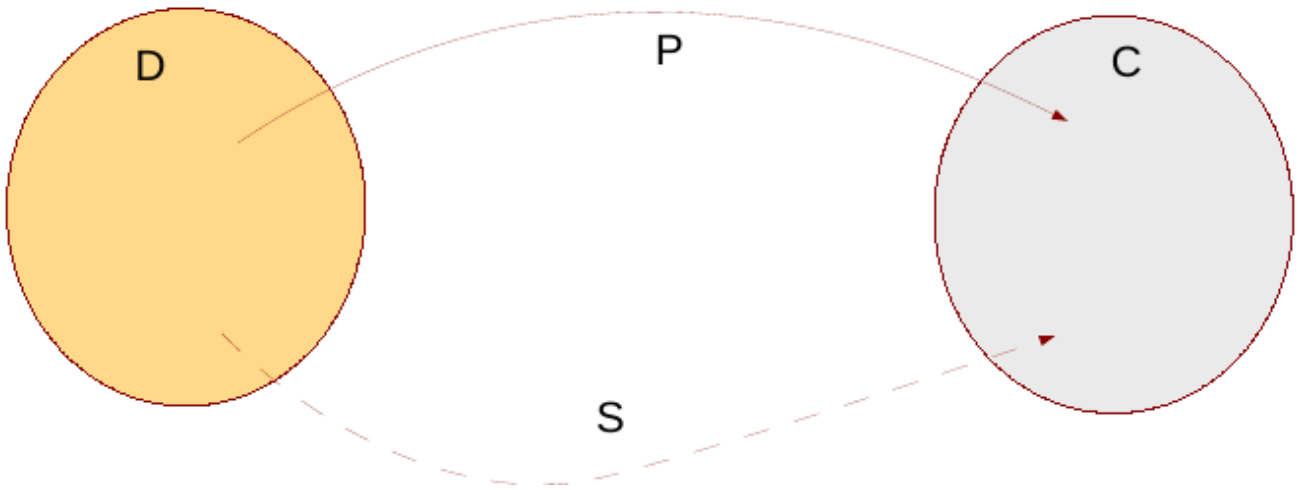
- **verifica** -> il prodotto è conforme alle specifiche?
  - conformità ai requisiti
  - esecuzione del giusto processo di realizzazione
  - in un certo momento dello sviluppo, si verifica che le specifiche fornite da qualcuno interno al processo di sviluppo siano rispettate
- **validazione** -> il prodotto che si sta sviluppando è effettivamente quello richiesto?
  - l'input è composto dai requisiti utente, che vengono applicati al prodotto sviluppato
  - si controlla che quanto fatto fino a un certo punto rispecchi le richieste dell'utente.

Tra gli obiettivi delle attività di V&V c'è rassicurare l'utilizzatore finale che il sistema sia adatto allo scopo richiesto.
Si intende fornire un **livello di fiducia**, il quale tuttavia dipende dal contesto, ossia da fattori quali:

- funzione del software all'interno dell'organizzazione
- attesa dell'utente
- politiche di mercato

Contesto generale

Sia S una descrizione di un comportamento, ossia una **specifica** da realizzare con un programma. In particolare, una specifica S è una descrizione che dichiara per gli input in D quali sono gli output attesi in C.



Un **programma** P è una funzione da un dominio D a un codominio C, ossia una particolare realizzazione di una specifica che tenta di essere più precisa di essa nella realizzazione delle funzionalità descritte da S.

Una tecnica di V&V cerca di stabilire se un programma P soddisfa una specifica S. Ciò avviene se e soltanto se $\forall\, d \in D\ P(d) = S(d)$. Questa condizione può essere verificata del tutto o approssimata. S può essere sia una specifica utente (quindi si parla di validazione), sia una specifica interna (si parla di verifica).

Approcci di V&V

Le tecniche di V&V possono essere utilizzate sia in verifica sia in validazione, a seconda del tipo di specifica:

- specifica di requisiti utente -> validazione
- specifica dei requisiti tecnici -> verifica

Esistono due approcci diametralmente opposti per la verifica della condizione di soddisfacimento di una specifica:

- **verifica formale**: si dimostra che il programma P soddisfa effettivamente la specifica S su ogni input
  - si rappresenta il programma come una rappresentazione matematica (grafi o clausole logiche) e si dimostrano dei teoremi su tale rappresentazione
    - ad esempio, si rappresenta il programma come un grafo e si verifica se esistono situazioni di deadlock
    - ancora, per i compilatori si può utilizzare una grammatica libera dal contesto (CFG) per verificarne il funzionamento, ossia verificare che il programma (visto come stringa) è derivabile dalla grammatica del linguaggio.
  - si effettuala verifica per ogni input possibile, oppure si trovano dei controesempi
  - approccio più auspicabile, ma non sempre fattibile
- **software testing and debugging**: si cercano di scoprire piccole imperfezioni nel programma P

- anziché verificare un'astrazione matematica di P, si effettuano le verifiche direttamente su P
- si accetta il rischio che P nasconda qualche imperfezione rispetto a S

Software testing e debugging

Il **software testing** è la pianificazione di strategie che evidenzino eventuali differenze tra P ed S.

Le attività atte a capire le cause delle differenze tra P ed S sono attività di **debugging**. Tramite debugging, è anche possibile risolvere tali anomalie.

Quanto detto fin'ora non è un processo, ma individua degli ambiti su cui si può lavorare.

Correttezza

Una possibile (e banale) definizione di **correttezza** è l'assenza di errori. Tuttavia, un errore è solo una delle possibili cause di un malfunzionamento. Pertanto, si preferisce disporre un sistema privo di malfunzionamenti.

Tradizionalmente, si distinguono tre concetti in relazione tra loro:

$$errore \implies difetto \implies malfunzionamento$$

Quando si parla di errore, si parla del motivo che ha introdotto un difetto all'interno del sistema, il quale potrebbe - potenzialmente - portare a un malfunzionamento. Un errore può essere di varia natura (organizzativo, implementativo ecc.).

Si può evitare che il sistema raggiunga delle failure con meccanismi di **defect containing** tramite meccanismi di **failure prevention**. Ad esempio, per Open Street Maps, si può implementare un meccanismo che, impostando il luogo di partenza o destinazione su un'isola, la controparte deve essere posizionata all'interno della stessa isola, per evitare di consigliare tragitti nell'acqua. In questo modo, non si risolvono le cause del problema, ma si evita che l'utente finale sperimenti il malfunzionamento.

D'altra parte, si possono utilizzare tecniche di **defect reduction** tramite **fault detection and removal**, che consistono nell'individuazione delle cause dei fault e la loro risoluzione.

Infine, esistono tecniche di **defect prevention**, come l'utilizzo di particolari pattern, architetture o framework. Sono le più difficile da gestire, perché prendono in considerazione sia le parti tecniche, sia le parti organizzative di chi può produrre l'errore.

Failure, fault ed error

Una **failure** (malfunzionamento) si riferisce al comportamento non corretto e osservabile del sistema software. In particolare, una failure è presente quando il sistema (visto come una scatola nera) non si comporta come previsto, a prescindere dalla sua implementazione.

Un **fault** (bug, difetto) si riferisce allo specifico codice o modello. È una condizione necessaria (ma non sufficiente) affinché si presenti una failure. Infatti, una failure deriva dall'interazione con il sistema, che non sempre può andare a toccare la parte che presenta un fault. Un fault può consistere di:

- circostanze non previste
- azione non correttamente implementate/modellate

Infine, un **errore** è la causa di un fault. Generalmente, è un errore umano (concettuale, di interpretazione, di digitazione ecc.).

Il software testing si occupa di individuare le failure, mentre il debugging individua i fault e cerca di rimuoverli, cercando anche di capire gli errori che li causano.

Esempio

Si consideri la seguente funzione

```
1. int makeItDouble(int param) {
2.   int result;
3.   result = param * param;  <^>
4.   return result
5. }
```

La chiamata `makeItDouble(3)` restituisce `9`.

- `9` rappresenta la failure (comportamento errato)
- la failure è causata dal fault in `<^>`
- possibili fonti dell'errore sono:
  - un typo: è stato digitato `*` al posto di `+`
  - un errore di interpretazione:
    - `double = param * 2`
    - `power = param * param`

D'altra parte, la chiamata `makeItDouble(2)` non evidenzia una failure, anzi fa sembrare che il programma funzioni correttamente.

Software testing

Il **software testing** consiste nell'esecuzione di alcuni **esperimenti** in un ambiente controllato al fine di acquisire sufficiente fiducia sul funzionamento. Di solito riguarda aspetti funzionali, ma può riguardare anche caratteristiche extra-funzionali.

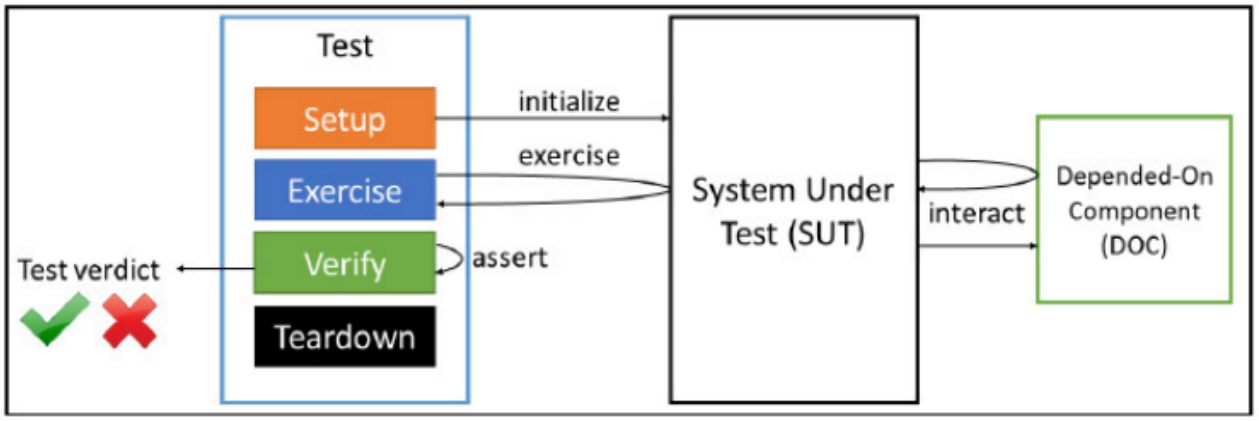
Vantaggi:

- prova che il sistema (quello reale, non la rappresentazione) risponde alle esigenze per cui era stato creato (avendo almeno un test per ogni requisito)
- fa manifestare eventuali malfunzionamenti
  Svantaggi:

> Il testing non può dimostrare l'assenza di guasti, ma solo la loro presenza ~ E.W. Dijkstra.

Un tipico contesto del software testing è composta da una procedura di test, in cui si effettua un setup, l'esecuzione di un'operazione (exercise) e la verifica delle asserzioni sul valore restituito dall'operazione.

- **inizializzazione** -> si inizializza il sistema under test
- **exercise** -> interazione della suit di test con il sistema inizializzato
- **verifica** -> controllo dei risultati ottenuti, asserendo delle condizioni; il passaggio positivo del test consiste nel match tra il risultato atteso e il risultato ricevuto dal sistema under test
- **teardown** -> si tira giù l'ambiente per l'under test



Esempio

Specifica: fattoriale su tutti gli interi. Il fattoriale di un negativo è da intendersi come il fattoriale del suo opposto.
Si esegue la seguente implementazione:

```
int myBuggyFact(int p) {
    int result = abs(p);
    if (result > 1){
        int next = result-1;
        result = result*myBuggyFact(next);
    }
    return result;
}
```

Si supponga di voler testare questo programma. Occorre chiedersi:

- quanto è ampio il dominio su cui fare il test?
- quanto è costoso esplorare l'intero dominio di input?
- si può considerare solo una sua sottoparte
- tutti i possibili input sono "equivalenti" tra loro?

Domande fondamentali del software testing

- Quali input selezionare?
  - Si possono scegliere valori rappresentativi, oppure partizionare il dominio di input e identificare delle classi di equivalenza rispetto a un obiettivo
- Quando smettere di testare?
  - Esistono delle definizioni di criteri di copertura (statement, archi, condizioni, flussi, funzionalità ecc.), così come valori minimi per l'accettazione
- Come si capisce se il risultato di un'operazione è corretto?
  - Si può creare un oracolo che restituisce sempre il risultato corretto (*oracle problem*), oppure lo si può dedurre da risultati storici del sistema o da sistemi equivalenti in esecuzione.

Correttezza e reliability

Per quanto detto fin'ora, la definizione iniziale di correttezza risulta essere troppo banale.

Si definisce dunque la **correttezza** come l'aderenza completa di un sistema alle sue specifiche. Essa viene asserita tramite prove formali, ossia ragionamenti logico/matematici. La validazione tramite testing richiederebbe l'esplorazione esaustiva del dominio di input.

D'altro lato, si definisce **affidabilità** (**reliability**) la stima della probabilità che un'operazione con il programma, a partire da un insieme di parametri di ingresso estratti casualmente dal dominio di input del programma, sia di successo. La reliability è un attributo statico del sistema, non un attributo di correttezza. In particolare, la reliability è pari a 1 meno la probabilità di errore (PFD, Probability of Failure on Demand).

Esempio

Si consideri il metodo `myBuggyFact` è si supponga che, nella pratica, il metodo verrà invocato in maniera equiprobabile con uno dei seguenti parametri: `[-5, -3, 0, 2, 4]`.

La PFD di `myBuggyFact` è 0.2, dunque la reliability è 0.8.

Profilo operazionale

Si introduce il **profilo operazionale**: è una descrizione numerica del modo o un insieme di modi di utilizzo del sistema. Ad esempio, il sistema viene utilizzato da queste tipologie di

utenti su queste funzionalità su queste percentuali di invocazioni di metodi e su queste percentuali di invocazioni di feature. In sostanza, è un insieme di descrizioni di come diversi attori interagiscono con le boundary del sistema e lo stimolano.

Ad esempio, per `myBuggyFactorial`, la reliability è $1 - p(0)$, dove $p(0)$ è la probabilità che venga inserito 0, ossia l' `80%`, vale a dire in questa percentuale di utilizzo il sistema apparirà funzionare correttamente.

Cambiando il profilo operazionale, cambia anche la percezione del sistema: esso permette di calcolare la reliability su un'idea di utilizzo del sistema, ma non la reliability in generale. In generale, la misura di "bontà" del sistema vale solo per il dominio di utilizzo definito dal profilo operazionale.

Sostanzialmente, un profilo operazionale è una descrizione numerica del sistema ed è la base di partenza per calcolare i diversi profili di reliability, ossia avere una confidenza dell'adeguatezza del sistema allo scenario ipotizzato. Averne uno aiuta nella pianificazione delle attività di test nei confronti degli utilizzatori, ma non sempre è utile basare una fase di test su di esso.

Si immagini, ad esempio, che ogni volta che una macchina finisca la benzina esploda. Un profilo operazionale sull'utilizzo tipico della macchina, ossia con il serbatoio pieno, non consentirebbe di testare il caso in cui il serbatoio si svuoti, lasciando un serio problema sconosciuto nel sistema.

Usare il profilo operazionale significa guardare l'utilizzo tipico del sistema in un certo contesto derivato dalla fase di analisi (happy scenario o worst case scenario).

Nel report del progetto, partendo dall'insieme di test progettati e sviluppati, si devono fare queste considerazioni sulla reliability del sistema al variare degli input inseriti.

Extra (dalla lezione successiva)

Le tecniche di verifica formale consente di avere una prova logico/matematica della correttezza di un sistema, o meglio di una sua rappresentazione logico/matematica (ad esempio un grafo o delle istruzioni formali). Tuttavia, questo dipende anche da come viene ricavata la rappresentazione.

D'altra parte, il software testing consente di scoprire gli errori in un sistema, ma non consente di affermarne l'assenza. Infatti, il testing consente di esplorare solo in un sottoinsieme del dominio di input, nella speranza che gli errori si trovino in una piccola sottoparte del dominio. Ci vuole bravura nel trovare questa sottoparte.

Se si utilizza il software testing, non si può ragionare della correttezza come l'assenza totale di errori nel sistema: aver passato una o più fasi del testing non assicura la sicurezza del sistema, dato che potrebbero esserci failure che non sono state esposte, non raggiungibili oppure è stato usato un sottoinsieme non adeguato del dominio di input.

Se non ci si trova in questi ultimi casi, si può parlare di **probabilità** che non si trovi un bug all'interno del sistema. In particolare, questa probabilità è una stima della reliability (attributo statistico): sotto certe condizioni il sistema aderisce alle specifiche con questa probabilità.

2 - Test automation and continuos testing

Software quality assurance

Si definisce **software quality assurance** l'insieme delle attività sistematiche che forniscono prova dell'idoneità all'uso concepito dell'intero prodotto software. In altre parole, è un processo nel quale si definiscono dei punti di controllo per misurare degli obiettivi (misurabili) da raggiungere.

Dalla definizione originale:

> "SQA is the systematic activities providing evidence of the fitness for use of the total software product."

- systematic activities -> SQA è un processo
- providing evidence -> sono necessari punti di controllo
- fitness -> si pongono degli obiettivi misurabili da raggiungere.

Tale processo, composto da piani e attività, è eseguito durante la fase di sviluppo del software per il monitoraggio e controllo dello sviluppo. L'obiettivo è assicurare, in un certo intervallo di confidenza, che il prodotto software:

- soddisfi le sue specifiche
- sia utilizzabile nel contesto richiesto
- sia sviluppato secondo le modalità stabilite

La creazione di un sistema software di qualità implica l'adozione in un intero processo che abbia l'obiettivo di assicurare la confidenza tra quanto si sta realizzando e le specifiche ad ogni passo. Ad esempio, si può prendere come specifica degli artefatti provenienti dall'analisi per andare incontro ad aspetti di validazione, oppure specifiche del modello di progettazione/implementazione per andare incontro ad aspetti di verifica.

La SQA è definita da tre componenti:

- **software testing** -> è solo uno degli aspetti del SQA, ma da solo non è sufficiente
- **quality control** -> pianificazione delle attività di controllo
- **software configuration management** -> pianificazione per la gestione degli artefatti software che si stanno elaborando.
  In particolare, gli strumenti di software testing non devono essere visti come standalone, ma sono integrati a tutte le fasi di sviluppo, dall'analisi al deployment.

Esempio

Si consideri di conoscere solo la firma del metodo `myBuggyFact`. Non sapendone l'implementazione, eseguire il testing su tutti i possibili input richiederebbe l'implementazione di 2^{32} casi di test. Invece, avendo a disposizione di un

Automated testing

Le attività di software testing sono tutte automatizzabili:

- generazione
- selezione
- prioritizzazione
- esecuzione
- report
- valutazione della bontà/validità di un dato insieme di casi d i test (trovare un oracolo per il sistema, applicare inferenza, utilizzare degli storici).

Queste attività automatizzate sono contrapposte a quelle fatte a mano dal team di QA e/o dall'utente finale (**manual testing**).

L'obiettivi dell'AT sono:

- migliorare l'efficienza dei processi di software testing
- migliorare l'efficacia dei processi di software testing

I benefici principali sono:

- riduzione dei costi
- riduzione della durata delle fasi di test
- aumento della reperibilità degli esperimenti considerati
- aumento dell'attendibilità statistica dei responsi delle fasi di QA
- miglioramento della documentazione delle failure.

Quality control

Il **quality control** è la definizione di un piano per il monitoraggio del processo di sviluppo e rilascio e il controllo del soddisfacimento dei requisiti. Gli strumenti utilizzati sono:

- il processo di Quality Assurance (QA)
- metodi di monitoraggio
- controlli di QA
  È necessaria che ci sia una definizione esplicita del processo di Quality Assurance accanto al processo di progettazione e sviluppo. Ad esempio, i requisiti dovranno essere scritti in un certo modo per poterli utilizzare nel piano di Quality Assurance per derivare i casi di test. Nella fase di analisi, devono essere prodotti un certo numero di documenti fatti in un certo modo da essere utilizzati come specifica nella fase di test.

Software configuration management

Il **software configuration management** è composto a sua volta da una serie di blocchi che indicano di sviluppare il software in un certo modo:

- **component identification** -> il software deve essere scritto in maniera più modulare possibile
- **version control** -> serve un meccanismo automatico di controllo delle versioni; senza questo, non è possibile fare SQA e testing in modo appropriato, poiché l'errore potrebbe non essere nel programma, ma nel modo i cui sono state messe insieme (manualmente) le versioni
- **configuration building** -> non è possibile creare un prodotto di qualità se sviluppato solo su una macchina, poiché il sistema stesso dipende dall'ambiente in cui è stato sviluppato; è necessaria una gestione delle configurazioni esplicita nelle fasi di build che astraggono il più possibile la configurazione del sistema dalla macchina.
- **change control**

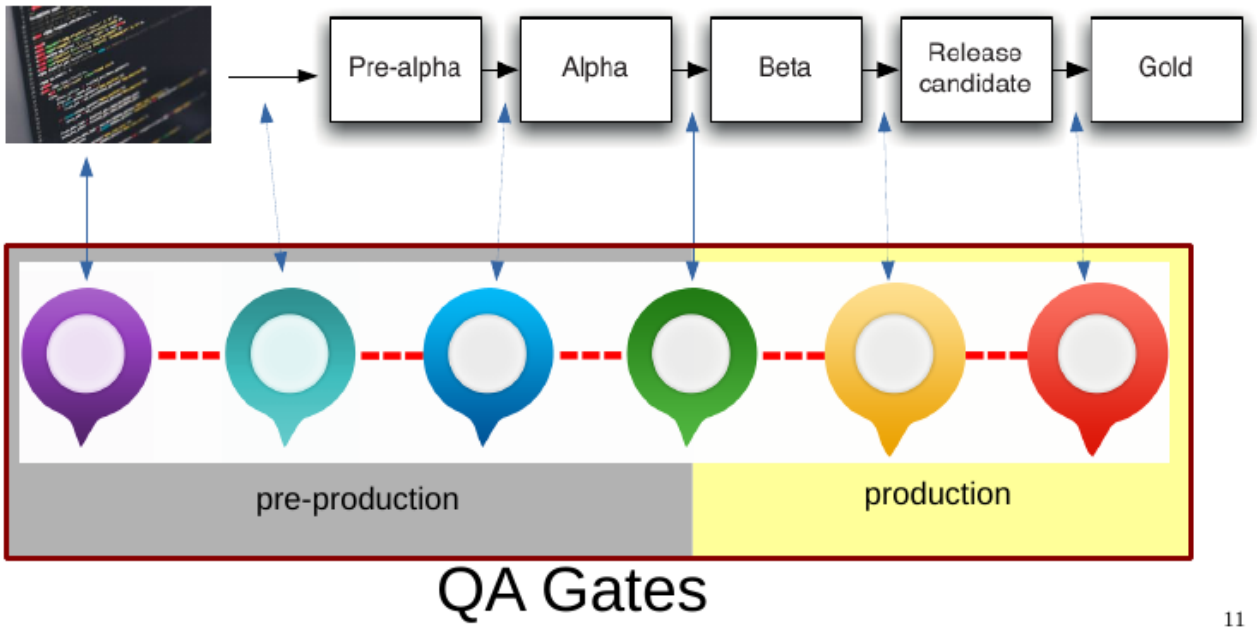
SQA e ciclo di rilascio - approccio tradizionale

È possibile immaginare il processo di quality assurance come una serie di tappe intervallate da dei **QA gates**, ossia obiettivi che indicano come deve essere fatto il prodotto fino a quel punto. Ogni QA gate definisce un momento di controllo con degli obiettivi misurabili, che possono riguardare la struttura del processo, gli outcome del processo o l'esecuzione del sistema software.

I QA gate scandiscono delle macroversioni del sistema. Le principali sono:

- **pre-aplha** -> sono stati raggiunti un certo grado di maturità, un certo grado di code smells, un certo numero di test, un certo grado di confidenza dei profili operazionali utilizzati per configurare il sistema ecc.
- **alpha** -> versione quasi finale del sistema testata dagli sviluppatori e dai provider in ambiente controllato
- **beta** -> versione rilasciata sul mercato e provata da un insieme ristretto di utilizzatori con i profili e le condizioni di utilizzo degli utilizzatori, ma non ancora la versione ufficiale
- **release candidate** -> versione che è quasi in uscita

- **gold** -> versione ufficiale



Nella pratica, succede spesso che il sistema sembri funzionare, ma appena rilasciato in ambiente operativo risulta non più funzionante. Ciò è probabilmente dovuto al fatto che il sistema è stato provato in un sottoinsieme di ambienti e scenari possibili. Il problema principale di questi tipi di scenari è vedere i QA gate in maniera discreta: la verifica degli obiettivi nel QA gate può andare a buon fine o meno. Tale discretizzazione, seppur utile, tende a creare questo tipo di problemi.

Nel tempo l'approccio è stato cambiato per percepire i controlli come un processo continuo e non più discreto. In particolare, ogni cambiamento deve essere percepito come una potenziale release del sistema. Ciò significa che tutte le volte che si aggiunge una funzionalità, il sistema software deve essere visto come un sistema complesso in cui il sistema di QA applica tutti i controlli previsti dall'inizio alla fine. Questo ha un maggiore costo in termini di soldi e tempo. Quando si sviluppa, occorre farlo in maniera intelligente e trasparente, in modo che ogni volta entrino in gioco i processo di QA in maniera automatica.

Automated testing

Per **automated testing** si intendono tutte le attività svolte per mezzo di infrastrutture o framework per il software testing che servono ad automatizzare:

- l'esecuzione e il report delle attività di test
- la selezione e la prioritizzazione dei test da eseguire
- la valutazione della bontà e della validità di un insieme di test case

L'automated testing è una parte di QA che porta automazione su ognuno di questi aspetti, cioè cercando di contribuire al claim *“every contribution leads to a potential release”*, quindi di cerca di contribuire in modo automatico per controllare in modo automatico che il software rispetti le specifiche. È possibile fare automated testing in diversi modi:

- strumenti che automatizzano i report e le attività di test -> ad ogni commit, vengono eseguiti in automatico tutti i test e viene generato un report
- altri strumenti generano in automatico test case da eseguire, oppure li selezionano o il prioritizzano (questo poiché lanciare i test può costare anche tanto)
- strumenti per valutare la bontà e la validità dei test case, che indicano quanto sono utili i test creati

L'automated testing si contrappone al testing svolto da umani (**manual testing**).

I principali obiettivi di AT sono migliorare l'efficienza e l'efficacia dei processi di QA. l'AT porta anche i seguenti benefici:

- riduzione dei costi e della durata delle fasi di test
- aumento della reperibilità degli esperimenti considerati
- aumento dell'attendibilità statistica dei responsi delle fasi di QA
- migliorare la documentazione delle failure.

L'automated testing si pone nel discorso *“every contribution leads to a potential release”* tramite **continuous testing**.

Continuous testing

Il **continuous testing** si occupa dell'integrazione dei processi di QA (in particolare dei processi di software testing) con:

- l'ambiente di sviluppo
- gli strumenti di build
- i sistemi di versioning

In particolare, si parla di strategia di testing in cui i test vengono eseguiti automaticamente sull'attuale versione del sistema, in particolare:

- su configurazioni locali allo sviluppo -> per far sì che il sistema funzioni localmente, al fine di continuare lo sviluppo
- su ambienti simili agli ambienti di produzione finali -> per garantirne il corretto funzionamento al rilascio

L'esecuzione dei test avviene in maniera trasparente. Infatti, gli sviluppatori non devono preoccuparsi di avviare esplicitamente le fasi di test, poiché questi vengono eseguiti dopo ogni comprovata evoluzione della code base.

I risultati de test sono notificati in modalità asincrona rispetto al termine dell'esecuzione, in modo da non interrompere lo sviluppo. Inoltre, è necessario prevedere delle policy che regolino quali test effettivamente eseguire: selezione, prioritizzazione, orchestrazione dei casi di test.

Integrazione di CI e CT

Si consideri un ciclo di svluppo definito dal seguente schema di build:

Lo sviluppatore sviluppa una nuova parte di codice e fa un commi. Il VCS riconosce il cambiamento di versione del sistema e genera un branch per le nuove funzionalità sviluppate dal dipendente e sulle quali eseguire i test. Allora, il VCS avvia una serie di attività di QA, in particolare di testing su un build server (server in grado di tirare su una serie di macchine virtuali con diversi SO). L'ambiente di testing è disaccoppiato da quello di sviluppo e punta ad essere il più simile possibile all'ambiente di deployment finale. Il testing genera un report che viene poi inviato allo sviluppatore, in modo che abbia conoscenza di eventuali problemi relativi alle funzionalità da lui sviluppate. Tutto ciò deve avvenire in maniera trasparente allo sviluppatore.

In particolare, il build server include:

- configurazione di un ambiente tipo
- build del sistema
- esecuzione dei test case
  Il test engineer deve strutturare il processo di CT che consenta allo sviluppatore in maniera indipendente al software testing.

Un simile approccio definisce quindi una tecnica di **CI + CT**:

- **CI** -> integrazione di pezzi di sistema diversi (si verifica se le funzionalità sviluppate sono ammissibili o meno)
- **CT** -> esecuzione continua di test sulla codebase (si fa partire - potenzialmente - l'intero processo di testing su tutta la codebase)

Queste attività sono eseguibili sono in un contesto di automated testing, ossia con avendo degli strumenti di VCS, generazione di test e report ecc.

Framework

Version control : Git

Git è il VCS più noto (ref). È open-source e distribuito.

L'importante è fare un branch del master su cui lavorare, poi fare il merge delle funzionalità sul master.

Diversi scenari:

- repository condiviso -> i vari sviluppatori eseguito i commit su un repository condiviso
- c'è un repository "benedetto" (o ufficiale della versione corrente del software come sorgente, ma non quello di release) che ha la responsabilità di mantenere la copia effettiva in sviluppo. Gli sviluppatori hanno delle copie di questo repository, su cui essi lavorano. È anche presente un gestore dell'integrazione (o integratore di sistema) che prende le parti sviluppate da ogni sviluppatore ed eventualmente ne fa il merge sul repository benedetto.
- in alcuni casi, l'integratore svolge la funzione di dittatore benevolo, cioè colui che controlla effettivamente lo sviluppo del sistema, ergo una modifica viene integrata solo se il dittatore decide che può essere integrata. I processi di QA possono prevedere nell'integratore dei test, ma esso può anche prevederne di suoi.
  Per eseguire i test, l'integratore crea un ulteriore fork della versione corrente su cui fa il merge del contributo. A quel punto, esegue i test ed eventualmente fa il merge nel ramo principale.

L'introduzione di un integratore è necessaria per avere un controllo (statico o dinamico) sull'integrazione e il testing del software.

Configuration building: Maven

Maven (ref) è uno strumento di build sviluppato da Apache Software Foundation. È uno strumento che si occupa di fare il build, compilare, aggregare ed eventualmente fare anche il deployment di sistemi software. Nasce principalmente per Java, ma è agnostico al sistema di sviluppo.

Maven implementa il paradigma del component-based development/design, ossia gestisce in maniera indipendente e transitiva l'import di librerie. È in grado infatti di recuperare in automatico le librerie di secondo livello necessarie a quelle di primo livello dichiarate dallo sviluppatore tramite degli appositi registry, aggiungendole anche al classpath in fase di compilazione.

Un progetto Maven è organizzato in un particolare schema di directory nel file system. Dal punto di vista logico sono delle directory collegate in un certo modo. La struttura delle directory è definita un **archetipo** (*archetype*). L'archetipo Java prevede la strutturazione in directory per i sorgenti e altre per i test.

Un progetto Maven può essere diviso in **moduli**, ossia separato in più parti che collaborano tra di loro, al fine di organizzare meglio la struttura del codice, la responsabilità di chi ci mette mano e i permessi e i processi di QA. I moduli possono essere dichiarati internamente o esternamente alla directory principale. I moduli sono esplicitati nella dichiarazione del progetto.

Tutte le informazioni di un progetto o modulo Maven sono raccolte in uno specifico descrittore, chiamato `pom.xml`. Questo file serve a battezzare il progetto/modulo, le dipendenze del progetto/modulo, quello che deve essere fatto nelle varie fasi di build, test, deploy del progetto/modulo. Tipicamente, il file introduce:

- artifactId
- groupId
- nome
- versione
- sviluppatori
- dipendenze
- direttive di build
  Spesso include anche direttive di configurazione, di test e di deploy.

L'output di un modulo Maven viene inserito in una directory chiamata `target`. In particolare, qui vengono inseriti tutti i risultati di una qualsiasi fase di Maven. Su un VCS non deve essere presente la directory `target`: se, ad esempio, i risultati in target vengono generati in parallelo da due sviluppatori su due macchine Linux e Windows, allora nell'eseguire i push ci saranno dei conflitti su quella cartella.

Nota: il comando `make clean` ripulisce la cartella `target` di tutti i risultati prodotti da Maven nelle sue fasi.

Maven non lavora solo localmente all'ambiente di sviluppo. Esso è un sistema distribuito di cui si scarica un client. Esiste un sistema distribuito globale di registri, chiamato **Maven Central**, che mantiene copie delle librerie Maven. Quando è necessaria una libreria non presente localmente:

- viene interrogata un'istanza di Maven Central (o un'altra specificata dall'utente)
- viene recuperata la copia richiesta della libreria
- tale copia viene salvata nella cache locale di Maven (directory `.m2`)
- viene inserito il path della libreria nel classpath per la compilazione

Maven è pensato per gestire l'intero processo di build. Questo è strutturato in fasi, concettualmente organizzate in sequenza. Tipicamente, errori nell'esecuzione di una fase comportano il fallimento dell'intero processo di build. Il risultato di una qualsiasi fase viene posto di default nella directory `target` del progetto. Le fasi di default di Maven sono:

- `validate` -> verifica che il progetto sia corretto e che tutte le informazioni necessarie siano disponibili
- `compile` -> compila il codice sorgente del progetto
- `test` -> testa il codice compilato utilizzando un framework di unit testing adatto
- `package` -> prende il codice compilato e lo impacchetta in un formato distribuibile, ad esempio JAR
- `integration-test` -> processa ed esegue il deployment il package se necessario in un ambiente in cui può essere eseguito l'integration testing
- `verify` -> esegue e controlla per verificare che il package sia valido e vada incontro ai criteri di qualità
- `install` -> installa il package nella repository locale, per utilizzarlo come dependency in altri progetti locali
- `deploy` -> eseguito in ambiente di integrazione o rilascio, copia il package finale alla repository remota
- `clean` -> pulisce gli artifact creati dall'ultima build dalla directory `target`
- `site` -> genera la site documentation per il progetto
  È anche possibile dichiarare ulteriori fasi.

Tutorial in 5 minutes. Reference.

Self-testing: JUnit

JUnit è un framework open source che supporta l'implementazione di test program in ambienti Java (ref). L'introduzione di JUnit ha contribuito:

- a rendere i test (specialmente di unità) una parte centrale nella fase di programmazione di un software
- allo sviluppo di tecniche di programmazione basate su testing (come TDD, Test Driven Development)

Si usa dire che JUnit è un DSL (Domain-Specific Language) che raffina Java inserendo concetti relativi al software testing, quali standard e dialetti specifici.

Tra i requisiti principali di JUnit figurano:

- supporto alla definizione di test case utili
- supporto alla definizione di test case ripetibili nel tempo
- riduzione del costo della scrittura dei test case attraverso pratiche orientate al riuso del codice

Tuttavia, JUnit non indica quale strategia di test usare, quali test siano più significativi o quali valori di input adottare o selezionare.

Disponibili diverse versioni:

- `4.x` -> la versione più diffusa
  - i test sono dichiarati per mezzo di annotazioni
  - sono previsti costrutti per il supporto del setup e il teardown del contesto di esecuzione
  - include strumenti per la gestione esplicita di come un insieme di test case debba essere eseguito (ossia, i **runners**)
- `5.x` -> la versione più recente
  - modularità dell'architettura aumentata
  - consente la composizione esplicita di più runners
  - differenzia due principali tipologie di use-case:
    - scrittura di programmi di test tramite l'invocazione di un'API
    - estensione della SPI (Service Provider Interface) per supportare diverse strategie per la scoperta/scelta ed esecuzione di test case

JUnit 4

In JUnit 4, un test case è un metodo pubblico annotato con `@Test`.

Le annotazioni `@Before` e `@After` indicano quali azioni eseguire prima e dopo ogni test:

- l'esecuzione di metodi annotati in questo modo non dipende dai risultati dei test
- non è specificato un ordine di esecuzione tra più metodi annotati con `@Before`, oppure tra metodi annotati con `@After`
  Solitamente, i metodi etichettati con `@Before` pongono delle precondizioni per i test, mentre quelli annotati con `@After` eseguono delle operazioni di cleanup a seguito dei test. Si può forzare JUnit per fare in modo che i metodi `@Before` e `@After` vengano eseguiti in maniera random per assicurarsi che non siano presenti bias dovuti alla macchina locale. Il fatto che l'ordine sia non deterministico è dovuto a motivi di prestazioni, essendo il tutto gestito tramite reflection e dipendendo dal modo in cui vengono popolate le strutture dati.

Le annotazioni `@BeforeClass` e `@AfterClass` indicano azioni da eseguire una sola volta prima o dopo l'esecuzione di tutti i metodi annotati con `@Test` all'interno della classe. Come per `@Before` e `@After`, non è specificato un ordine di esecuzione. Questi metodi sono statici.

Se si utilizza JUnit 4 insieme a Maven, i nomi delle classi di test devono rispecchiare i seguenti pattern:

- `Test*`

- `*Test`
- `*Tests`
- `*TestCase`
  Infatti, per gli archetipi Java, Maven riconosce le classi di test in base al pattern del nome della classe.

Continuous integration

Piattaforme per la continuous integrazione e la continuous delivery sono:

- TRAVIS-CI (ref)
- GitHub Actions (ref)
- GitLab CI-CD (ref)
- Circle-CI (ref)

I framework di CI sono offerti come servizi remoti e prevedono policy e costi di utilizzo:

- spesso non ci sono costi per progetti open-source
- se presenti, le limitazioni di utilizzo imposte su account freeware sono spesso sovrabbondanti per gli scopi del corso e non è richiesta alcuna sottoscrizione a pagamento

In generale, ogni framework di CI ha le sue specifiche procedure di configurazione rispetto al VCS utilizzo.

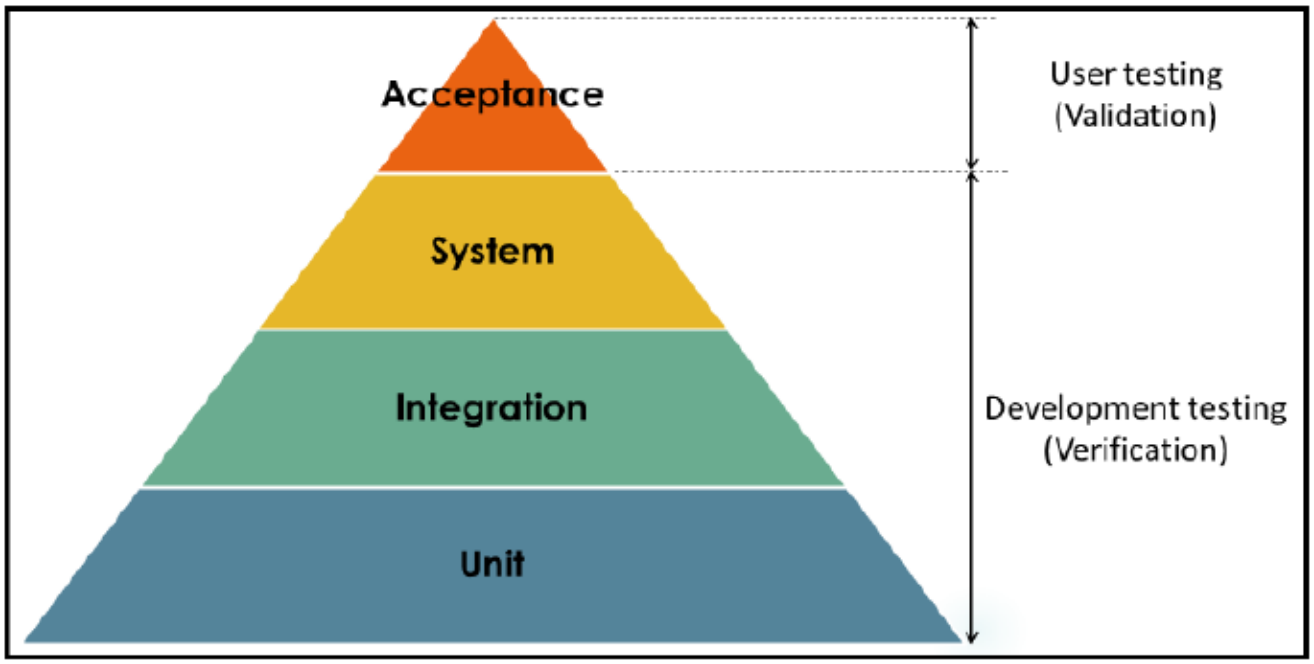
3 - Unit and integration testing - concetti

Aspetti del software testing

Quando si parla di ST, si possono considerare tre aspetti fondamentali:

- **livello** -> contesto degli obiettivi
  - unità: c'è una stretta vicinanza tra il programmatore/progettista dell'unità e il programmatore/progettista dei test, persone che spesso coincidono; si prende una porzione atomica del system under test (metodo, package, classe) e chi ne ha scritto il codice scrive dei test per verificare che tali funzionalità rispecchino le specifiche
  - integrazione: dopo i test di unità, si prendono più moduli del sistema per verificare che funzionino in cooperazione tra loro
  - sistema: si fa integrazione a livello di sistema, ossia test di integrazione con tutti i moduli del sistema
  - accettazione: è estremamente vicino all'idea di validazione; si valida il sistema rispetto ai requisiti utente, ergo si valuta se i committenti sono soddisfatti o no
- **metodo**
  - black-box (non si conosce l'implementazione del codice che si testa, ma solo specifiche in linguaggio naturale o modelli come SD/AD; si verifica che la scatola nera rispetta questi aspetti)
    - svantaggi: se gli errori non sono tracciabili da aspetti funzionali, ma solo dall'implementazione, trovare input per cui i test falliscono può risultare più costoso; è un metodo più difficile, poiché si basa sul *cosa* testare
    - vantaggi: questo metodo si basa sulle specifiche, quindi sulle funzionalità, astraendo le funzionalità
  - white-box (si conosce l'implementazione del codice che si vuole testare; si possono ignorare anche le specifiche, dato che questo metodo è guidato dalla specifica dell'implementazione)
    - svantaggi: il testing è guidato dall'implementazione, che potrebbe anche essere completamente sconnessa dai requisiti
    - vantaggi: è più semplice, poiché si basa sul *come* testare (dipende dal linguaggio); consente automazione più efficiente
  - non funzionale: si vede come viene svolta un'attività funzionale, come tempo di esecuzione, sicurezza, manutenibilità.
- **tipo**
  - manuale
  - automatico (automazione, generazione dei casi di test, prioritizzazione)

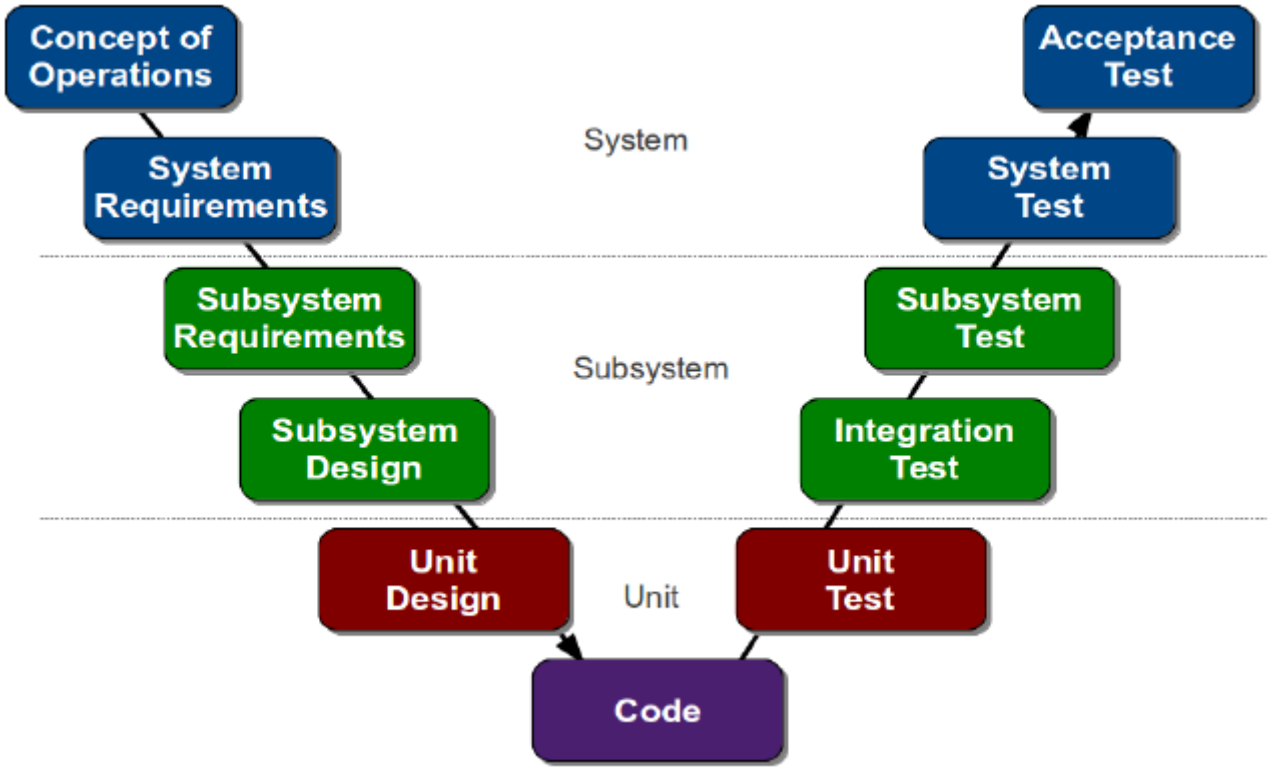
Testing e V&V



Nei test di unità, integrazione e sistema si fa verifica, mentre i test di accettazione comportano validazione.

Notare la differenza delle aree della piramide: in un sistema software si tendono ad avere molti test di unità, dato che i programmatori che sviluppano i test scrivono molti casi di test; man mano che si sale di livello, è sempre più difficile trovare casi di test sensati. Inoltre, gli strumenti di supporto riflettono - più o meno - questa piramide: ci sono molti strumenti di supporto per i test di unità, meno per quelli di accettazione.

V-model



Un'altra vista che si può avere dei test è il **v-model**, che schematizza il rapporto tra le attività di sviluppo e quelle di V&V. Le fasi di sviluppo sono semplificate in una cascata, ma vanno viste come uno spin di uno sviluppo iterativo.

Il fulcro di questo schema è che ad ogni fase dello sviluppo dovrebbe corrispondere un'attività parallela specifica di V&V. Ad esempio, quando si progetta un sottosistema, si dovrebbero pensare dei relativi test di integrazione.

Nel dettagli:

1. il designer specifica e implementa delle funzionalità
2. il designer rilascia le specifiche al tester
3. il tester specifica e implementa i test
4. il test esegue i casi di test

L'esecuzione automatica dei casi di test può avvenire:

- ad ogni proposta di modifica pervenuta (come un commit o una pull-request)
- al rilascio di una nuova funzionalità
- al rilascio di una nuova versione

Test di unità

Gli obiettivi dei testi di unità sono:

- rilevare malfunzionamenti su un singolo modulo o unità funzionale (metodo, classe, package, ma non un modulo Maven). La granularità del modulo dipende dal contesto.
- acquisire confidenza che tale modulo del SUT segua il comportamento atteso se eseguito in isolamento

Solitamente i test di unità sono implementati dagli stessi sviluppatori che hanno implementato il modulo in questione, anche prima della scrittura del codice (Test Driven Development). **Nota**: il TDD può includere anche l'aggiunta di casi di test a seguito della scrittura del codice; in breve, è un processo iterativo.

Si possono stabilire l'adeguatezza e la bontà dei test di unità in funzione:

- del numero di funzionalità controllate, del numero dei requisiti controllati, del numero di aspetti rilevabili dalle specifiche -> si stabilisce se i casi di test sono buoni rispetto alle funzionalità rispetto al *cosa* (specifiche, requisiti)
- dalle metriche di copertura del codice sorgente a disposizione-> si stabilisce se i casi di test sono buoni rispetto alle funzionalità rispetto al *come* (implementazione)

JUnit

JUnit è un framework nato originariamente per sviluppare l'implementazione di test di unità in ambienti Java.

Requisiti principali:

- supportare la definizione di casi di test utili
- supportare la definizione di casi di test ripetibili nel tempo
- ridurre i costi di scrittura dei casi di test tramite pratiche orientate al riuso del codice

Esempio

Uno o più casi di test corrispondono a una classe, che definisce il "runner". I metodi di test (annotati con `@Test`) corrispondono all'implementazione di una sequenza di passi che specificano il test. I test possono essere parametrici, ossia indipendenti dai valori specifici utilizzati. I parametri possono essere passati (in JUnit 4) tramite un metodo pubblico e statico che restituisce una collezione di oggetti. Questo metodo può avere qualsiasi nome e serve a codificare i parametri su cui effettuare i test, che vengono scelti dal progettista dei casi di test. Il metodo per l'ottenimento dei parametri viene chiamato prima dell'istanziazione della classe, che viene istanziata una volta per ogni elemento nella collezione tramite il costruttore (non di default).
(spiegare slide 10)

Dentro JUnit, tutto questo lavoro è eseguito dal runner, in questo caso la classe `Parameterized` di JUnit, che esegue questi passi nella sequenza indicata.

Pre-testing e post-testing

Esistono annotazione per eseguire operazioni una sola volta prima o dopo l'esecuzione di tutti i casi di test di una classe, quali `@BerforeClass` e `@AfterClass`. Non è specificato un ordine di esecuzione tra più metodi annotati con la stessa annotazione.

Esempio

(riportare slide 12)

È responsabilità del designer dei casi di test l'identificazione dettagliata dei valori attesi da un test, attraversi valori puntuali o la consultazione di un oracolo (funzione di predizione o sistema esterno).

È responsabilità del programmatore dei casi di test implementare le scelte progettuali attraverso un insieme di istruzioni di verifica e controllo (come `Assert`).

Quando si progettano i casi di test, occorre definire il valore atteso dei test.

Test di integrazione

I test di integrazione puntano a scovare problemi dovuto all'interazione tra diversi moduli del sistema software. In particolare, si vuole acquisire confidenza che:

- ci siano le dovute interconnessioni tra moduli/componenti del SUT
- le interazioni tra i moduli del SUT rispettano i "contratti funzionali" stabiliti da protocolli/specifiche di interazione, ossia nella maniera progettata

Ad esempio, se una classe A aggrega le classi B e C, il test di integrazione verifica sia che un'istanza di A sia collegata ad oggetti di classe B e C, sia che sia connessa agli oggetti corretti di B e C (ad esempio, si può verificare che lo studente sia connesso al proprio corso di laurea e non uno qualsiasi).

I test di integrazione assumono che ogni modulo funzioni correttamente in isolamento, ossia ha passato tutti i test di unità.

Chi esegue i test di integrazione dipende dal caso specifico: possono essere sviluppatori, integratori, tester.

I test di integrazione si possono eseguire secondi diversi approcci:

- white-box: si redaggono i test avendo a disposizione il codice dei moduli da testare
- black-box: si conosce solo la specifica delle interfacce esposte dai moduli
  Nella pratica, si tiene conto di entrambi gli approcci contemporaneamente.

Nei test di integrazione, spesso si ha la necessità di utilizzare degli **stub** (o **mock**), elementi software definiti in fase di test (come una classe, un modulo, un'unità ecc.) che non è soggetto a test di per sé - pur facendo parte dell'ambiente di test - e mima il comportamento dell'unità mancante, spesso attraverso comportamenti banali o costanti. In questo modo, il test viene eseguito in modo incrementale, considerando di volta in volta parti aggiuntive del SUT, simulando quelle al momento non disponibili.

I test di integrazione hanno anche bisogno di **test driver** avanzati per:

- configurare l'ambiente di test
- coordinare le risorse
- eseguire il clean-up dopo l'esecuzione

Strategie per i test di integrazione

In generale, trovare la soluzione ottima è un problema difficile (NP-completo).

I test di integrazione si possono svolgere seguendo diverse strategie:

1. big bang -> si integrano tutte le componenti insieme e ne si testa il funzionamento
   - svantaggi: è difficile e non consente di individuare tutti i problemi
2. top-down -> si esegue l'interazione delle interfacce dei moduli

- i test che si definiscono sono volti a verificare che le connessioni tra le componenti siano corrette e che lo siano anche i dati passati
- è utile usare questo approccio quando si ha una chiara definizione delle interfacce dei moduli
- spesso gli scenari sono definiti a partire dagli scenari d'uso del SUT
- si utilizzano gli stub per simulare il comportamento o le funzionalità mancanti
- esempio:
    1. si testa C1, usando degli stub per C2 e C3
    2. si testano C1 e C2, usando uno stub per C3
    3. si testano C1, C2 e C3, usando uno stub per C4
    4. si testano C1, C2, C3 e C4, usando uno stub per C5
    5. si testano tutte le classi

3. bottom-up -> si parte dallo strato profondo del sistema, fino ad arrivare ai bordi (boundary)
    - si usano i test driver per simulare il comportamento dei moduli o componenti più esterni del sistema
    - anche qui le funzionalità mancanti sono simulate tramite stub
    - si raffina considerando interfacce più generali o focalizzandosi sulle parti più periferiche del sistema
    - esempio:
        1. si testa C5
        2. si testano C5 e C4 (che "stimola" C5 come test driver)
        3. si testano C5, C4 e C2
        4. si testano C5, C4, C3 e C2
        5. si testano tutte le classi

4 - Strumenti per test di unità e integrazione

Alcuni degli strumenti più noti per l'esecuzione di test sono:

- JUnit come test driver
- Mockito per la creazione di stub/mock (ref)

Mockito

Mockito è un framework per la definizione di stub a supporto del testing in Java.

Fornisce un insieme di primitive che consentono di:

- dichiarare stub/mock
- definire valori di ritorno per operazioni su specifiche istanze
- (ri)definire comportamenti per operazioni in base a specifiche istanze in input

Supporta nativamente paradigmi di programmazione agile guidati dalla scrittura di casi di test, quali:

- Test Driven Development (TDD)
- Behavior Drive Development (BDD)

Alcuni costrutti chiave di Mockito sono:

- creazione programmatica dei mock/stub:
  - `mock()` -> altera il comportamento della classe, aggiungendo o rimuovendo funzionalità, anche corner case (**Nota**: non è bene definire mock che interagiscano ad ogni scenario, bensì definirne di diversi per ogni scenario possibile)
  - `spy()` -> una certa istanza di un'unità è osservata e si vuole verificare se ne viene invocato un metodo, modificato lo stato ecc.
- dichiarazioni semplificate per mezzo di annotazioni sintattiche:
  - `@Mock`
  - `@Spy`
  - `@Captor`
  - `@InjectMocks` -> l'istanza annotata in questo modo verrà considerata da Mockito per creare al suo interno dei mock, ossia fare il mock di alcuni suoi attributi (specificati in fase di definizione dei test); la differenza è che, in questo caso, non si vuole fare il mock di tutte le istanze di una classe, ma solo delle istanze che compongono gli attributi della classe annotata con `@InjectMocks`

Esempio:

```
// Let's import Mockito staticallu so that the code looks clearer
import static org.mockito.Mockito.*;

// Mock creation
List mockedList = mock(List.class);

// You can mock concrete classes, not just interfaces
LinkedList mockedLinkedList = mock(LinkedList.class);
```

Nota: quando si ha a che fare con singleton o attributi statici, possono essere creati effetti indesiderati dall'esecuzione di un test che vanno a impattare altri test, venendo meno alla loro indipendenza. In questi casi, vanno definiti dei metodi per rimuovere questi elementi.

- definizione programmatica dei valori di ritorno
  - `Mockito.when(mockedInstance.fooOperation()).thenReturn(true)` -> cattura la chiamata a `fooOperation()` di `mockedInstance` e sostituisce la vera implementazione il valore di ritorno definito in `thenReturn`, ossia `true`
  - `BDDMockito.given(mickedInstance.fooOperation()).willReturn(true)`
- programmazione personalizzata dei comportamenti
  - `Mockito.when(mockedInstance.fooOperation()).thenAnswer(new Answer<Object>() { ... })` -> al posto dei `...` si mette una nuova implementazione personalizzata del metodo `fooOperation`, detta Answer, che viene sostituita da Mockito nella chiamata all'implementazione vera e propria del metodo
- verifica programmatica se i mock sono stati usati/riferiti
  - `Mockito.verify(mockedInstance).fooOperation()` -> si verifica che il metodo `fooOperation` sia stato chiamato
  - `BDDMockito.then(mockedInstance).should().fooOperation()`
- integrazione con JUnit:
  - `MockitoJUnitRunner`
  - `MockitoJUnit.rule()`

5 - Approcci alla generazione di test

L'esecuzione di casi di test non può confermare l'assenza di errori da un sistema. La generazione di casi di test consiste nel pianificare degli esperimenti per interagire con il sistema creato. Chiaramente, non è possibile interagire con il sistema con tutte le possibili configurazioni (principalmente per motivi di budget), quindi deve cercare di raggiungere un certo grado di confidenza con il sistema, selezionando appositamente le configurazioni necessarie e/o possibili. Ci si pongono in particolare tre domande:

- quali e quanti input selezionare?
- quando smettere di testare?
- come si può sapere se il risultato di un'operazione è corretto?

Domande

Quali e quanti input selezionare?

L'esplorazione esaustiva del dominio di input è spessi impraticabile (ad esempio la funzione fattoriale). Esistono diversi approcci per la selezione degli input:

- uso di dati da interazioni passate: si utilizzano gli stessi dati utilizzati in interazioni passate
  - vantaggi:
    - i dati passati possono ripresentarsi in futuro
    - si testato modi di utilizzo effettivamente sperimentati nel sistema
    - ottimo metodo per focalizzarsi sui casi più sollecitati
    - utile nelle attività che guardano alla validazione del sistema
    - può dare indicazioni sulla reliability effettiva del sistema in operazione
  - svantaggi:
    - se il progetto o la funzionalità sono nuovi, non esistono storici di dati
    - i dati passati potrebbero non coprire tutti input possibili del sistema, che possono cambiare nel tempo
    - potrebbe esservi un alto numero di casi di test dagli storici
- selezione randomica degli input: si scelgono degli input a caso e si testa il sistema con questi:
  - vantaggi:
    - semplicità nella realizzazione del metodo (benché apparente, dato che il metodo potrebbe essere vincolato a valori specifici)
    - utile nelle fasi iniziali dei test per rilevare malfunzionamenti grossolani
  - svantaggi:

        - la ricerca è cieca
        - gli input trovati potrebbero non essere rappresentativi e sarebbe necessario un alto numero di casi di test per raggiungere un livello di confidenza adeguato
- generazione automatica rispetto a un obiettivo di adequacy: si creano degli strumenti che, in maniera automatica, esplorano il dominio di input e cercano di ottimizzare una funzione obiettivo, in questo caso l'**adequacy** (misura della bontà di un insieme di test)
    - vantaggi:
        - altamente automatizzabile in molti casi
        - facilmente integrabile con approcci di sviluppo e CI
    - svantaggi:
        - nei casi in cui la generazione è guidata dal SUT, possono risultare molti test poco significativi per le effettive condizioni di utilizzo del sistema
        - nei casi in cui la generazione è guidata da modelli del SUT, si hanno strumenti di generazione meno versatili nell'integrazione con i processi di sviluppo
        - complessità legate alla scelta degli iperparametri e alle euristiche di generazione
- partizionamento del dominio in classi di equivalenza rispetto gli input atteso e allo stato del SUT: si possono riconoscere dei cluster di input "simili", in modo da scegliere un rappresentante per cluster:
    - vantaggi:
        - indipendente dall'implementazione
        - nella fase di progettazione dei test, di cerca di identificare aspetti salienti del dominio o modi di utilizzo effettivamente sperimentati nel sistema
    - svantaggi:
        - la qualità dei test è in funzione dei livelli di chiarezza dei requisiti e delle specifiche
        - si riescono a prendere in considerazione solo vincoli o limitazioni esplicitamente espressi
        - familiarità del SUT
        - conoscenza dei contesti operativi e di deployment
- interazione con strumenti di intelligenza artificiale (LLM): si sfruttano forme di AI addestrate, interagendo con loro tramite linguaggio naturale:
    - vantaggi:
        - semplicità (per quanto apparente) di interazione con gli LLM
        - grande potenza di ragionamento in tempi relativamente ristretti
        - onniscenza (anch'essa apparente) degli LLM
    - svantaggi:
        - soluzioni in funzione della formulazione del problema (prompt engineering)
        - eccessiva fiducia negli LLM, dato che le risposte osservate potrebbero essere frutto di bias accidentali

- ragionamento black-box e probabilistico (gli outcome non possono essere considerati deterministici)

Quando smettere di testare?

Come si può sapere se il risultato di un'operazione è corretto?

Una volta scelti gli elementi significativi del dominio di input, è necessario assegnare ad ogni input un output atteso dal sistema. Questo si può fare in diverse modi:

- tramite un oracolo che risponde sempre correttamente
- utilizzando dei valori storici, specifiche, sistemi equivalenti, inferenze su proprietà ecc.

La specifica dei casi di test deve prevedere l'output per ogni interazione con il sistema o elemento testato. Inoltre, è possibile avere dei casi di test che prevedano fallimenti del sistema, ad esempio tramite lancio di un'eccezione o un errore.

Nota: la scelta dei valori di input del sistema è solo una parte della specifica del test. Affinché questa sia completa, occorre aggiungere l'output atteso dal sistema.

Approcci

Randoop

Randoop è un generatore randomico di unit test per programmi Java, che crea automaticamente dei programmi JUnit che implementano dei casi di test.

Può essere usato per la generazione dei test per il progetto.

Classi di equivalenza

Un dominio di input può essere partizionato in **classi di equivalenza**, in modo tale che, per ogni elemento in una partizione, il SUT deve mostrare lo stesso comportamento, o quantomeno si può assumere che sia così dalla specifiche.

Nella test suite in costruzione T_g deve essere incluso un elemento per ogni partizione. In particolare, la qualità di T_g sarà dovuta a:

- chiarezza dei requisiti e delle specifiche: vincoli o limitazioni esplicitamente espressi
- familiarità del SUT: conoscenza dei contesti operativi e di deployment
- conoscenza dell'implementazione: dettagli su come sono realizzate procedure e controlli

(continua nelle slide successive).

6 - Partizione del dominio

Le classi di equivalenza consentono di ridurre il dominio per il testing a un insieme finito e trattabile di classi.

Linee guida generali per le classi di equivalenza

Per le attività di test, si possono considerare uno o più parametri di input:

- direttamente derivanti dal dominio del sistema (key abstraction, feature, necessità, requisiti)
- esplicitato al solo livello di astrazione considerato (funzione di una particolare scelta implementativa)

Si può partizionare lo spazio dei valori ammissibili basandosi alla conoscenza esplicita o implicita di:

- **range** (impliciti o espliciti): considerare un valore nel range e due valori al di fuori, uno per ogni lato del range
- **stringhe**: considerare almeno un insieme di stringhe tutte valide, un insieme di stringhe tutte non valide, un insieme di stringhe vuote e un insieme di stringhe nulle
- **enumerazioni**: considerare ogni valore assegnato a una classe di equivalenza differente
- **array**: considerare almeno una partizione con array tutti legali, una con array tutti vuoti e una dove tutti gli array superano la lunghezza massima consentita
- **tipi di dato complessi**: considerare partizioni applicando iterativamente i criteri generali ai dati nella struttura; considerare i valori nulli.

Queste linee guida sono generali ed è giusto considerarle, ma non in modo esclusivo. Spesso, classi di equivalenza significative possono essere dedotte dalla **semantica** del parametro/attributo/variabile considerata. Ad esempio, un attributo password si tipo stringa può essere una stringa vuota, `null`, valida nel dominio considerato (e di conseguenza, corretta o errata per l'utente) oppure non valida nel dominio corretto.

I concetti di validità e correttezza dipendono dal dominio considerato:

- un valore assegnato a un parametro è **valido** se rispetta le regole del dominio; il valore è **non valido** se non le rispetta
- un valore valido è **corretto** in base alla specifica configurazione del SUT.

Ad esempio, si consideri l'istruzione:

```
User u = DAOUser.getInstance().retrieveUser(email);
```

Si possono avere i seguenti scenari:

- `email="-11jud7@sjlkla9873@jjdud."` non è un valore valido per il parametro `email`
- `email="mrfoo@nothing.org"` è un valore valido, ma sua correttezza dipende dal SUT: infatti, l'utente con questa email potrebbe esistere o anche no.

Criteri per identificare i test

L'identificazione dei test passa per la combinazione di due metodologie: unidimensionale e multidimensionale.

Nell'approccio **multidimensionale**, si considera il prodotto cartesiano dei parametri di input e si scelgono i test coprendo la matrice delle combinazioni delle classi di equivalenza, ossia si sceglie un rappresentate per ogni classe di equivalenza (dimensione) e si effettuano le varie combinazioni.

Nell'approccio **unidimensionale**, non si provano tutte le combinazioni di parametri, bensì, per ogni classe di equivalenza considerata, si fa in modo che esista un test che rappresenti quella classe di equivalenza.

Procedura sistematica per il partizionamento del dominio

1. identificare i domini di input
   - dalla definizione di key abstraction, feature, needs, requisiti
   - dai tipi di dato legati a particolari scelte implementative
   - da condizioni imposte (esplicitamente o implicitamente)
2. identificare le classi di equivalenza per ogni parametro del test
3. combinare le classi di equivalenza
4. eliminare le combinazioni di classi di equivalenza non ammissibili

Esempio

Si consideri l'operazione nella classe `LedgerHandler` di BookKeeper

```
void asyncReadEntriesInternal(long firstEntry, long lastEntry,
ReadCallback cb, Object ctx, boolean isRecoveryRead)
```

Nota: per semplicità di sta considerando che:

- il SUT sia un singolo metodo; in generale, questo non avviene, ma viene considerato come SUT in intera classe, un package, un modulo ecc.

Si comincia a fare le classi di equivalenza in base al tipo di dato:

- `isRecoveryRead` : {true}, {false}
- `cb` : {null}, {"valid_instance"}, {"invalid_instance"}
- `ctx` : {null}, {"valid_instance"}, {"invalid_instance"}
- `firstEntry` : {<=0}, {>0}
- `lastEntry` : {<=0}, {>0}
- `LedgerHandle` : {vuoto}, {ha entry sufficienti}, {non ha entry sufficienti}

In questo caso, le dimensioni sono rappresentate dai parametri formali del metodo under test, più lo stato della classe che lo contiene.

Si possono definire classi di equivalenza meno grossolane?
Si può ragionare sul fatto che `firstEntry` e `lastEntry` devono essere relazionate:

- `firstEntry` : {<=0}, {>0}
- `lastEntry` : {< `firstEntry` }, {= `firstEntry` }, {> `firstEntry` }
  Si possono raffinare anche le classi di equivalenza di `LedgerHandle` :
- `LedgerHandle` : {#istanze < `lastEntry-firstEntry` }, {#istanze >= `lastEntry-firstEntry` }

Nota: non è accettabile escludere i casi di input "invalid_instance" per i tipi di dato complesso, giustificando la scelta con considerazioni del tipo "Il costruttore ritorna solo istanze corrette". Infatti, il codice del costruttore potrebbe cambiare, introducendo bug, e soluzioni polimorfe potrebbero consentire di legare istanze di sottoclassi che fanno riferimento a implementazioni del costruttore diverse da quelle considerate.
Un'istanza invalida è creabile con un mock, oppure estendendo la classe considerata e facendo in modo che restituisca degli errori.

Una volta definite e combinate le classi di equivalenza, occorre trovare un rappresentante per ognuna. Teoricamente, posto che le classi siano corrette, scegliere un rappresentante qualunque può andare bene. Nella pratica, è più probabile che l'errore
si trovi ai margini delle classi di equivalenza e quindi si scelgono valori ai bordi delle classi.

Si considerino ad esempio `firstEntry` e `lastEntry` , lasciando fissi gli altri parametri. In base al dominio e alla conoscenza del SUT, si possono scegliere non solo valori di boundary, ma anche valore un po' più dentro al bordo. Si hanno dunque i seguenti boundary values:

- `firstEntry` : -1; 0; 1
- `lastEntry` : `firstEntry-1` ; `firstEntry` ; `firstEntry+1`

Per l'approccio multidimensionale, si hanno le seguenti combinazioni: (ricopiare da slide 40).

Large Language Models

I LLM sono implementazioni di grandi reti neurali, costruiti secondo specifiche architetture di riferimento, addestrati su enormi quantità di dati utilizzando approcci supervisionati.

Uno dei loro punti di forza principali è nell'interazione uomo-macchina in linguaggio naturale:

- i linguaggi di programmazione sono in genere liberi dal contesto e possono essere considerati come un sottoinsieme ristretto dell'insieme dei linguaggi naturali
- sfruttare le capacità dei LLM di elaborare informazioni da e per specifici linguaggi di programmazione

Nel SE, si possono sfruttare le capacità di elaborazione dei LLM per supportare aspetti di generazione automatica di artefatti software; in particolare, nel software testing, la generazione automatica di software test.

Prompting

L'interazione con un LLM avviene tramite una richiesta in linguaggio naturale, detta **prompt**. È possibile definire una specifica struttura dei prompt da sottoporre: questa è abbastanza arbitraria e può riguardare aspetti sintattici, semantici, conversazionali e di contesto